

INTRODUCTION TO EMU8086\ASSEMBLY

MAHLA ENTEZARI

Introduction to EMU8086\Assembly
Mahla Entezari

انواع عملوندها:

رجیسترها: (Registers)

رجیسترها بخش‌های مهمی از پردازنده هستند که برای ذخیره‌سازی داده‌ها و اجرای عملیات‌ها استفاده می‌شوند. در Emu8086 ، رجیسترها به دو دسته اصلی تقسیم می‌شوند:

• رجیسترهای ۱۶ بیتی:

- AX رجیستر انبار (Accumulator)
- BX رجیستر پایه (Base)
- CX رجیستر شمارنده (Count)
- DX رجیستر داده (Data)

• رجیسترهای ۸ بیتی:

- AX به دو بخش AH (بخش بالا) و AL (بخش پایین) تقسیم می‌شود.
- BX به دو بخش BH (بخش بالا) و BL (بخش پایین) تقسیم می‌شود.
- CX به دو بخش CH (بخش بالا) و CL (بخش پایین) تقسیم می‌شود.
- DX به دو بخش DH (بخش بالا) و DL (بخش پایین) تقسیم می‌شود.

• رجیسترهای عمومی: (General Purpose Registers)

- AX, BX, CX, DX این رجیسترها به طور کلی برای عملیات ریاضی و منطقی مورد استفاده قرار می‌گیرند.

• رجیسترهای نمایه‌گر: (Index Registers)

- SI (Source Index), DI (Destination Index) برای عملیات مرتبط با آرایه‌ها و رشته‌ها استفاده می‌شوند.

• رجیسترهای پشتیبانی: (Segment Registers)

- CS (Code Segment), DS (Data Segment), ES (Extra Segment), SS (Stack Segment) این رجیسترها برای مدیریت حافظه و دسترسی به حافظه استفاده می‌شوند.

حافظه: (Memory)

در زبان اسمبلی، می‌توان به مقادیر حافظه دسترسی پیدا کرده و با آنها کار کنیم.

عملوند درجا: (Immediate Operand)

عملوندهای درجا مقادیر ثابتی هستند که مستقیماً در دستورالعمل‌ها استفاده می‌شوند

سیگنال‌ها: (Flags)

سیگنال‌ها برای نشان دادن وضعیت‌های مختلف عملیات‌ها استفاده می‌شوند:

1. **ZF (Zero Flag):** نشان می‌دهد که نتیجه عملیات صفر است.
2. **SF (Sign Flag):** نشان می‌دهد که نتیجه عملیات منفی است.
3. **CF (Carry Flag):** نشان می‌دهد که عملیات دارای رقم نقلی بوده است.
4. **OF (Overflow Flag):** نشان می‌دهد که عملیات سرریز شده است.
5. **PF (Parity Flag):** نشان می‌دهد که تعداد بیت‌های یک در نتیجه زوج است.

دستورات ریاضی و منطقی

Emu8086 دارای مجموعه‌ای از دستورات ریاضی و منطقی است که شامل عملیات جمع، تفریق، ضرب، تقسیم و عملیات منطقی مانند AND، OR، XOR و غیره می‌شود.

- کاهش عملوند به اندازه ۱: (DEC)
- افزایش عملوند به اندازه ۱: (INC)
- خاتمه برنامه: (HLT)
- تقسیم عدد علامت‌دار: (IDIV)
- ضرب عدد علامت‌دار: (IMUL)
- وقفه: (INT)
- پرش در صورت برابر بودن: (JE)
- پرش بدون شرط: (JMP)
- ذخیره آدرس در رجیستر: (LEA)
- شبیه‌سازی حلقه: (LOOP)
- جابجایی اطلاعات بین رجیسترها و حافظه: (MOV)
- ضرب عدد مثبت: (MUL)

برخی از دستورات اصلی عبارتند از:

• جمع و تفریق:

ADD destination, source
SUB destination, source

ترکیب‌های عملوندی ممکن:

- رجیستر و حافظه (REG, memory)
- حافظه و رجیستر (memory, REG)
- رجیستر و رجیستر (REG, REG)
- حافظه و مقدار درجا (memory, immediate)
- رجیستر و مقدار درجا (REG, immediate)

به عنوان مثال:

ADD AX, BX ; $AX = AX + BX$
SUB CX, DX ; $CX = CX - DX$

• ضرب و تقسیم:

MUL operand
DIV operand

به عنوان مثال:

MUL BX ; $AX = AX * BX$
DIV CX ; $AX = AX / CX$ (DX و باقیمانده در AX)

• عملیات منطقی:

AND destination, source
OR destination, source
XOR destination, source

به عنوان مثال:

AND AX, BX ; $AX = AX \text{ AND } BX$

ترکیب‌های عملوندی ممکن:

- REG, memory ○
- memory, REG ○
- REG, REG ○
- memory, immediate ○
- REG, immediate ○

OR CX, DX ; $CX = CX \text{ OR } DX$
XOR SI, DI ; $SI = SI \text{ XOR } DI$

• مقایسه دو عملوند: (CMP)

CMP AX, BX ;
ترکیب‌های عملوندی ممکن:

- REG, memory ○
- memory, REG ○
- REG, REG ○
- memory, immediate ○
- REG, immediate ○

نمونه کد

نمونه‌ای از کد اسمبلی برای Emu8086 برای جمع دو عدد در رجیسترهای AX و BX به صورت زیر است:

MOV AX, 5 ; AX = 5

MOV BX, 3 ; BX = 3

ADD AX, BX ; AX = AX + BX (AX = 8 now)

این کد ساده نشان می‌دهد که چگونه از دستور MOV برای انتقال مقدار به رجیسترها و دستور ADD برای انجام عملیات جمع استفاده می‌شود.

Mahla Entezari

Introduction to EMU8086\Assembly